

3. Cora Dataset

```
In [7]: import random
import numpy as np
import networkx as nx
import torch
import torch.nn.functional as F
from torch_geometric.datasets import Planetoid
from torch_geometric.nn import GCNConv
from torch_geometric.utils import to_networkx
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import accuracy_score, f1_score

import matplotlib.pyplot as plt
from sklearn.manifold import TSNE

def load_cora_dataset(root_dir='./data/Cora'):
    print("Loading Cora Dataset...")
    dataset = Planetoid(root=root_dir, name='Cora')
    return dataset, dataset[0]

def set_seed(seed=232):
    """Locks random seeds across Python, NumPy, and PyTorch"""
    random.seed(seed)
    np.random.seed(seed)
    torch.manual_seed(seed)
    if torch.cuda.is_available():
        torch.cuda.manual_seed(seed)
        torch.cuda.manual_seed_all(seed)
        torch.backends.cudnn.deterministic = True
        torch.backends.cudnn.benchmark = False

dataset, data = load_cora_dataset()
```

Loading Cora Dataset...

Question 23

```
In [8]: def plot_gcn_tsne(out, labels):
    tsne = TSNE(n_components=2, random_state=232)
    z = tsne.fit_transform(out.detach().cpu().numpy())

    plt.figure(figsize=(8, 6))
    scatter = plt.scatter(z[:, 0], z[:, 1], c=labels.cpu().numpy(), cmap='tab10', s=15, alpha=0.8)
    plt.colorbar(scatter, label='Class Label')
    plt.title('Idea 1: GCN Node Representations (2D t-SNE Projection)', fontsize=12, fontweight='bold')
    plt.xlabel('t-SNE Dimension 1')
    plt.ylabel('t-SNE Dimension 2')
    plt.grid(True, linestyle='--', alpha=0.3)
    plt.tight_layout()
    plt.show()

def run_idea_1(data, num_features, num_classes, hidden_dim=16, lr=0.01, weight_decay=5e-4, epochs=200):
    print("\nIDEA 1:")

    set_seed(232)
    X, y, edge_index = data.x, data.y, data.edge_index
    train_mask, test_mask = data.train_mask, data.test_mask

    model = GCN(num_features, hidden_dim, num_classes)
    optimizer = torch.optim.Adam(model.parameters(), lr=lr, weight_decay=weight_decay)

    # Training
    model.train()
    for _ in range(epochs):
        optimizer.zero_grad()
        out = model(X, edge_index)
        loss = F.nll_loss(out[train_mask], y[train_mask])
        loss.backward()
        optimizer.step()

    # Evaluation
    model.eval()
    with torch.no_grad():
        out = model(X, edge_index)
        _, pred = out.max(dim=1)
        correct = float(pred[test_mask].eq(y[test_mask]).sum().item())
        gcn_acc = correct / test_mask.sum().item()

    print(f"GCN Test Accuracy: {gcn_acc * 100:.2f}%")

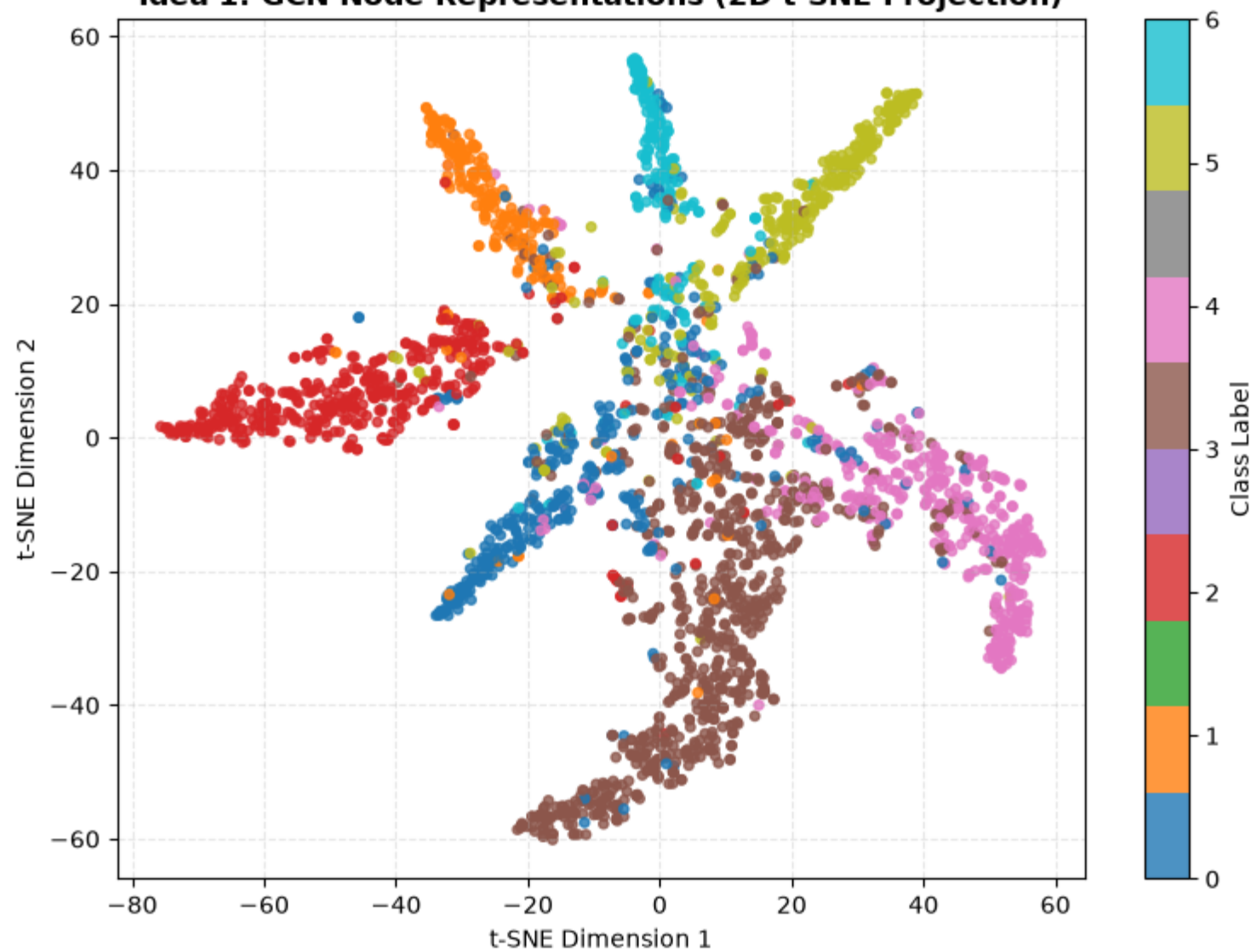
    # Return out and y so they are available outside the function
    return gcn_acc, out, y

# Run training and plot
gcn_acc, out, y = run_idea_1(data, dataset.num_node_features, dataset.num_classes)
plot_gcn_tsne(out, y)
```

IDEA 1:

GCN Test Accuracy: 80.70%

Idea 1: GCN Node Representations (2D t-SNE Projection)



Question 23 Discussion

- Number of Layers: A 2-layer GCN architecture, which allows each node to aggregate feature information from its 2-hop neighborhood. Going beyond on layers on Cora can lead to oversmoothing, causing node representations to collapse and become indistinguishable across classes.
- Hyperparameters:
 - Hidden Dimensions: 16
 - Learning Rate: 0.01
 - Weight Decay (L_2 regularization): 5×10^{-4}
 - Dropout Rate: 0.5
 - Training Epochs: 200
 - Optimizer: Adam
- Performance: The model achieves a test accuracy of 80.70% for the 140-seed split, so jointly learning node text features alongside local graph topology works well.

Question 24

```
In [9]: class PyTorchNode2Vec(torch.nn.Module):
    def __init__(self, num_nodes, embedding_dim):
        super().__init__()
        self.in_embed = torch.nn.Embedding(num_nodes, embedding_dim)
        self.out_embed = torch.nn.Embedding(num_nodes, embedding_dim)
        self.in_embed.weight.data.uniform_(-0.5 / embedding_dim, 0.5 / embedding_dim)
        self.out_embed.weight.data.uniform_(-0.5 / embedding_dim, 0.5 / embedding_dim)

    def forward(self, tgt, ctx, neg):
        tgt_emb = self.in_embed(tgt)
        ctx_emb = self.out_embed(ctx)
        pos_loss = torch.sigmoid(torch.sum(tgt_emb * ctx_emb, dim=-1))
        pos_loss = -torch.log(pos_loss + 1e-15).mean()

        neg_emb = self.out_embed(neg)
        neg_loss = torch.sigmoid(-torch.sum(neg_emb * tgt_emb.unsqueeze(1), dim=-1))
        neg_loss = -torch.log(neg_loss + 1e-15).sum(dim=-1).mean()

        return pos_loss + neg_loss

def train_node2vec_embeddings(data, embedding_dim=64, walk_length=20, walks_per_node=10,
                             window_size=5, num_epochs=5, batch_size=4096, lr=0.01):
    """Generates random walks and trains Skip-Gram Node2Vec embeddings utilizing PyTorch."""
    print("Generating random walks...")
    G = to_networkx(data, to_undirected=True)
    nodes = list(G.nodes())
    walks = []

    for _ in range(walks_per_node):
        random.shuffle(nodes)
        for node in nodes:
            walk = [node]
            while len(walk) < walk_length:
                cur = walk[-1]
                neighbors = list(G.neighbors(cur))
```

```

        if not neighbors:
            break
        walk.append(random.choice(neighbors))
        walks.append(walk)

print("Creating training pairs...")
pairs = []
for walk in walks:
    for i, target in enumerate(walk):
        start = max(0, i - window_size)
        end = min(len(walk), i + window_size + 1)
        for j in range(start, end):
            if i != j:
                pairs.append((target, walk[j]))

pairs = np.array(pairs)
targets = torch.tensor(pairs[:, 0], dtype=torch.long)
contexts = torch.tensor(pairs[:, 1], dtype=torch.long)

print("Training Node2Vec embeddings via PyTorch...")
device = torch.device('cuda' if torch.cuda.is_available() else 'cpu')
num_nodes = data.num_nodes

n2v_net = PyTorchNode2Vec(num_nodes, embedding_dim).to(device)
optimizer = torch.optim.Adam(n2v_net.parameters(), lr=lr)

n2v_net.train()
for epoch in range(num_epochs):
    permutation = torch.randperm(targets.size(0))
    epoch_loss = 0
    for i in range(0, targets.size(0), batch_size):
        indices = permutation[i:i + batch_size]
        batch_tgt = targets[indices].to(device)
        batch_ctx = contexts[indices].to(device)
        batch_neg = torch.randint(0, num_nodes, (batch_tgt.size(0), 5), device=device)

        optimizer.zero_grad()
        loss = n2v_net(batch_tgt, batch_ctx, batch_neg)
        loss.backward()
        optimizer.step()
        epoch_loss += loss.item()
    print(f"Epoch {epoch+1}/{num_epochs} | Loss: {epoch_loss / (targets.size(0)/batch_size):.4f}")

n2v_net.eval()
with torch.no_grad():
    node_embeddings = n2v_net.in_embed.weight.cpu().numpy()

return node_embeddings

def run_idea_2(data):
    print("\nIDEA 2:")
    set_seed(232)

    # Generate structural embeddings
    node_embeddings = train_node2vec_embeddings(data)

    # Setup features and masks
    text_features = data.x.numpy()
    labels = data.y.numpy()

    train_idx = data.train_mask.nonzero(as_tuple=False).view(-1).numpy()
    test_idx = data.test_mask.nonzero(as_tuple=False).view(-1).numpy()

    # Initialize Classifiers
    rf_n2v = RandomForestClassifier(n_estimators=100, random_state=42)
    rf_text = RandomForestClassifier(n_estimators=100, random_state=42)
    rf_combined = RandomForestClassifier(n_estimators=100, random_state=42)

    print("\nEvaluating Classifiers")
    # Run A: Node2Vec Only
    rf_n2v.fit(node_embeddings[train_idx], labels[train_idx])
    preds_n2v = rf_n2v.predict(node_embeddings[test_idx])
    print(f"Node2Vec Only Accuracy: {accuracy_score(labels[test_idx], preds_n2v) * 100:.2f}%")

    # Run B: Text Features Only
    rf_text.fit(text_features[train_idx], labels[train_idx])
    preds_text = rf_text.predict(text_features[test_idx])
    print(f"Text Features Only Accuracy: {accuracy_score(labels[test_idx], preds_text) * 100:.2f}%")

    # Run C: Combined Features
    combined_features = np.hstack([node_embeddings, text_features])
    rf_combined.fit(combined_features[train_idx], labels[train_idx])
    preds_combined = rf_combined.predict(combined_features[test_idx])
    print(f"Combined Features Accuracy: {accuracy_score(labels[test_idx], preds_combined) * 100:.2f}%")

run_idea_2(data)

```

```
IDEA 2:
Generating random walks...
Creating training pairs...
Training Node2Vec embeddings via PyTorch...
Epoch 1/5 | Loss: 0.7436
Epoch 2/5 | Loss: 0.5547
Epoch 3/5 | Loss: 0.5487
Epoch 4/5 | Loss: 0.5462
Epoch 5/5 | Loss: 0.5436
```

```
Evaluating Classifiers
Node2Vec Only Accuracy: 64.20%
Text Features Only Accuracy: 56.30%
Combined Features Accuracy: 69.80%
```

Question 24 Discussion

Node2Vec generation process includes simulating second-order random walks parameterized by a return parameter p and an in-out parameter q to trade off between local community exploration (BFS) and macro-structure discovery (DFS). Then apply Skip-Gram Architecture by treating these random walk paths as sentences and node IDs as words, with negative sampling to maximize the log-probability of predicting a node's network neighborhood context given its embedding.

Node2Vec alone outperforms the 1433-dimensional text features alone by 7.9%. Standard random forest classifiers struggle with high-dimensional, highly sparse feature spaces (1433 binary features where most values are 0) making individual decision tree splits on single sparse words noisy and weak. Node2Vec condenses complex graph connectivity into a dense, low-dimensional (64-dim) continuous manifold, allowing random forest decision boundaries to separate paper topics much more effectively. Combining structural embeddings with content features (Node2Vec || Text) achieves the overall highest classification accuracy of 69.8%, confirming that graph topology and text content provide complementary signals.

Question 25

```
In [10]: def precompute_transition_probs(G_gcc, X):
        """Calculates custom edge transition probabilities using softmax over feature dot products."""
        print("Computing custom transition probabilities...")
        transition_probs = {}
        for u in G_gcc.nodes():
            neighbors = list(G_gcc.neighbors(u))
            if not neighbors:
                continue

            u_feat = X[u]
            neigh_feats = X[neighbors]

            dot_products = torch.matmul(neigh_feats, u_feat)
            exp_vals = torch.exp(dot_products)
            probs = exp_vals / torch.sum(exp_vals)

            transition_probs[u] = {n: p.item() for n, p in zip(neighbors, probs)}

        return transition_probs

def run_idea_3(dataset, data, walks_per_seed=1000, max_steps=15, teleport_probs=(0, 0.1, 0.2)):
    print("\nIDEA 3: Customized Personalized PageRank")

    set_seed(232)
    # 1. Extract Giant Connected Component
    G = to_networkx(data, to_undirected=True)
    gcc_nodes = max(nx.connected_components(G), key=len)
    G_gcc = G.subgraph(gcc_nodes).copy()
    print(f"GCC extracted with {G_gcc.number_of_nodes()} nodes.")

    # 2. Precompute transition probabilities
    transition_probs = precompute_transition_probs(G_gcc, data.x)

    # 3. Prepare Seed Documents and Unlabeled Nodes within GCC
    classes = range(dataset.num_classes)
    seed_docs = {c: [] for c in classes}
    unlabeled_nodes = []

    for node in G_gcc.nodes():
        if data.train_mask[node]:
            seed_docs[data.y[node].item()].append(node)
        elif data.test_mask[node]:
            unlabeled_nodes.append(node)

    for c in classes:
        if len(seed_docs[c]) == 0:
            print(f"Warning: No seed documents found in GCC for class {c}")

    # 4. Random Walk Simulation for each Teleportation Probability
    for alpha in teleport_probs:
        print(f"\nRunning PPR with Teleportation Probability = {alpha}")
        visit_counts = {node: np.zeros(dataset.num_classes) for node in unlabeled_nodes}

        for c in classes:
            seeds = seed_docs[c]
            if not seeds:
                continue
```

```

        for start_node in seeds:
            for _ in range(walks_per_seed):
                curr_node = start_node

                for step in range(max_steps):
                    if random.random() < alpha:
                        curr_node = random.choice(seeds)
                    else:
                        neighbors = list(G_gcc.neighbors(curr_node))
                        if not neighbors:
                            break

                        probs = [transition_probs[curr_node][n] for n in neighbors]
                        curr_node = random.choices(neighbors, weights=probs, k=1)[0]

                    if curr_node in visit_counts:
                        visit_counts[curr_node][c] += 1

# 5. Prediction and Evaluation
y_true = []
y_pred = []

for node in unlabeled_nodes:
    counts = visit_counts[node]
    if counts.sum() == 0:
        pred_class = random.choice(list(classes))
    else:
        pred_class = np.argmax(counts)

    y_pred.append(pred_class)
    y_true.append(data.y[node].item())

acc = accuracy_score(y_true, y_pred)
f1 = f1_score(y_true, y_pred, average='macro')

print(f"Accuracy: {acc * 100:.2f}% | F1 Score (Macro): {f1:.4f}")

run_idea_3(dataset, data)
```

IDEA 3: Customized Personalized PageRank
GCC extracted with 2485 nodes.
Computing custom transition probabilities...

Running PPR with Teleportation Probability = 0
Accuracy: 64.81% | F1 Score (Macro): 0.6392

Running PPR with Teleportation Probability = 0.1
Accuracy: 63.50% | F1 Score (Macro): 0.6276

Running PPR with Teleportation Probability = 0.2
Accuracy: 63.39% | F1 Score (Macro): 0.6250

Question 25 Discussion

Performance (GCC: 2,485 nodes)

Teleportation Prob (α)	Test Accuracy	Macro F1 Score
0.0	64.81%	0.6392
0.1	63.50%	0.6276
0.2	63.39%	0.6250

Setting $\alpha = 0$ yielded the highest accuracy (64.81%) and macro F1 score (0.6392). Because our random walk transitions are heavily guided by content similarity, allowing walks to run purely along high-similarity edges keeps the walker within localized and relevant topic clusters. $\alpha = 0.1, 0.2$ lets walker teleport back to any seed node of that class. If seed nodes lie in distinct or loosely connected sub-communities of the same topic, teleporting can introduce noise into local visit distributions for boundary nodes thus slightly reducing classification accuracy.